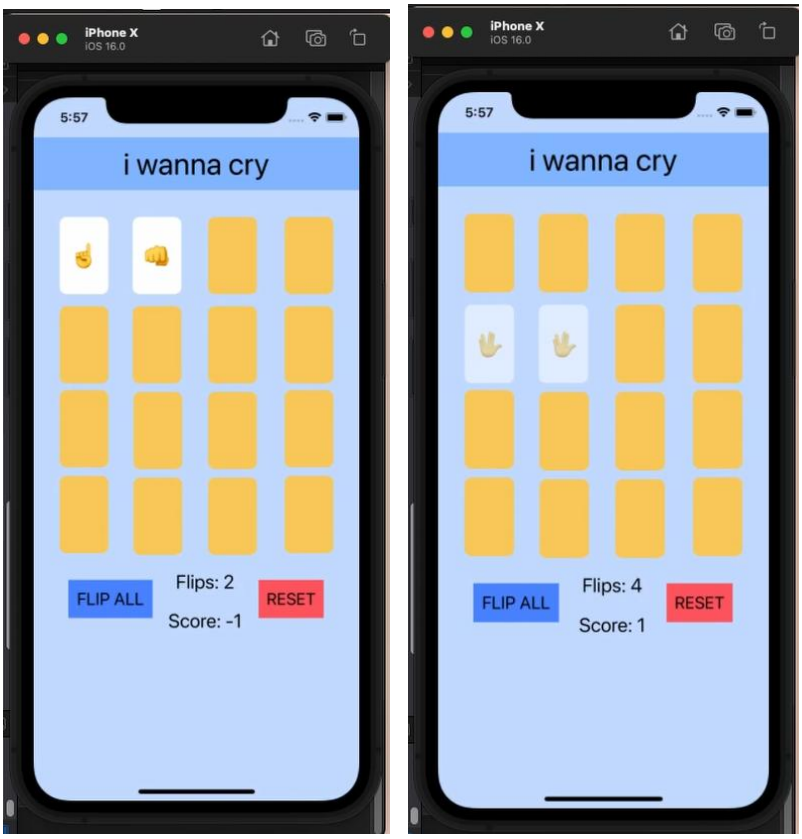
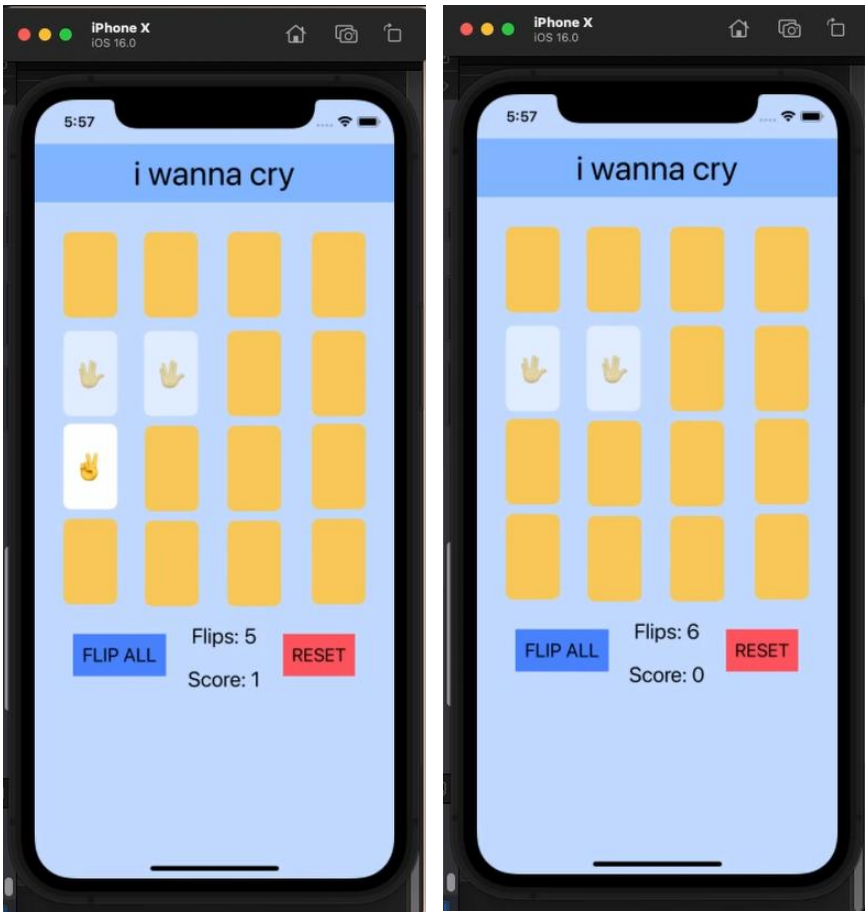
資工三 112590019 謝政軒

Lab4 - Card Matching Game

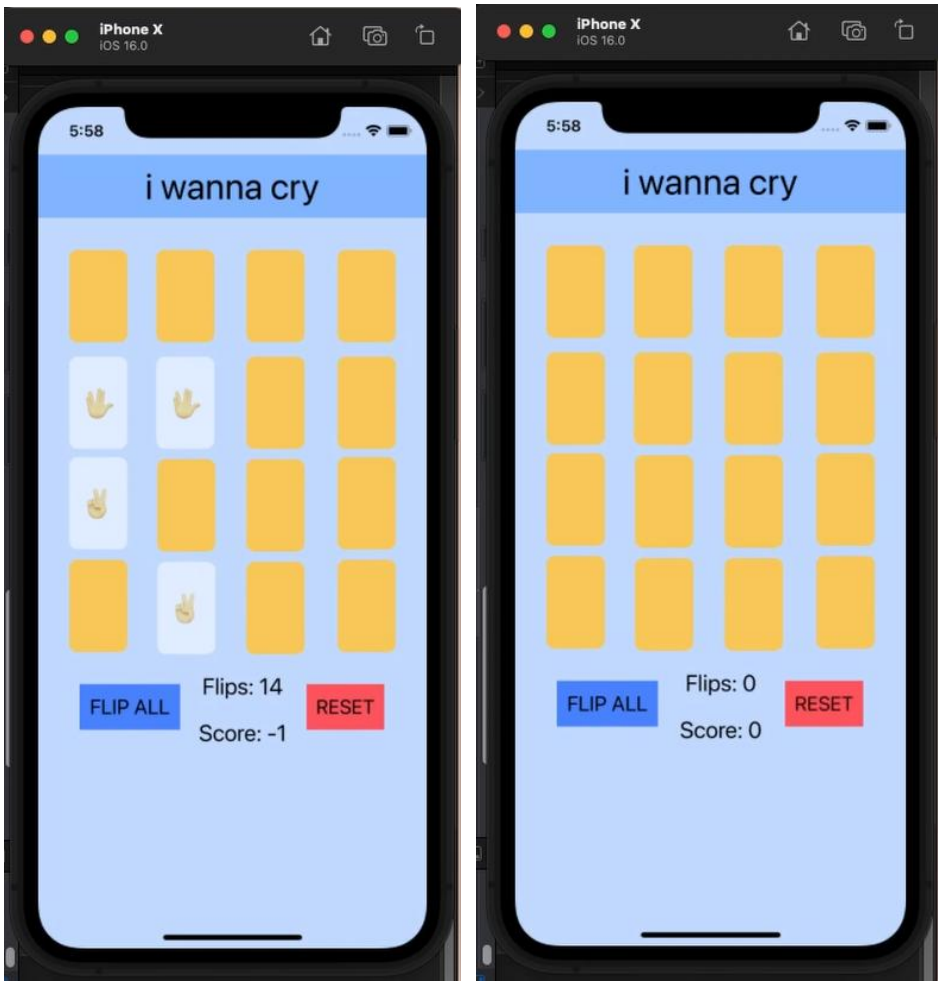
功能：



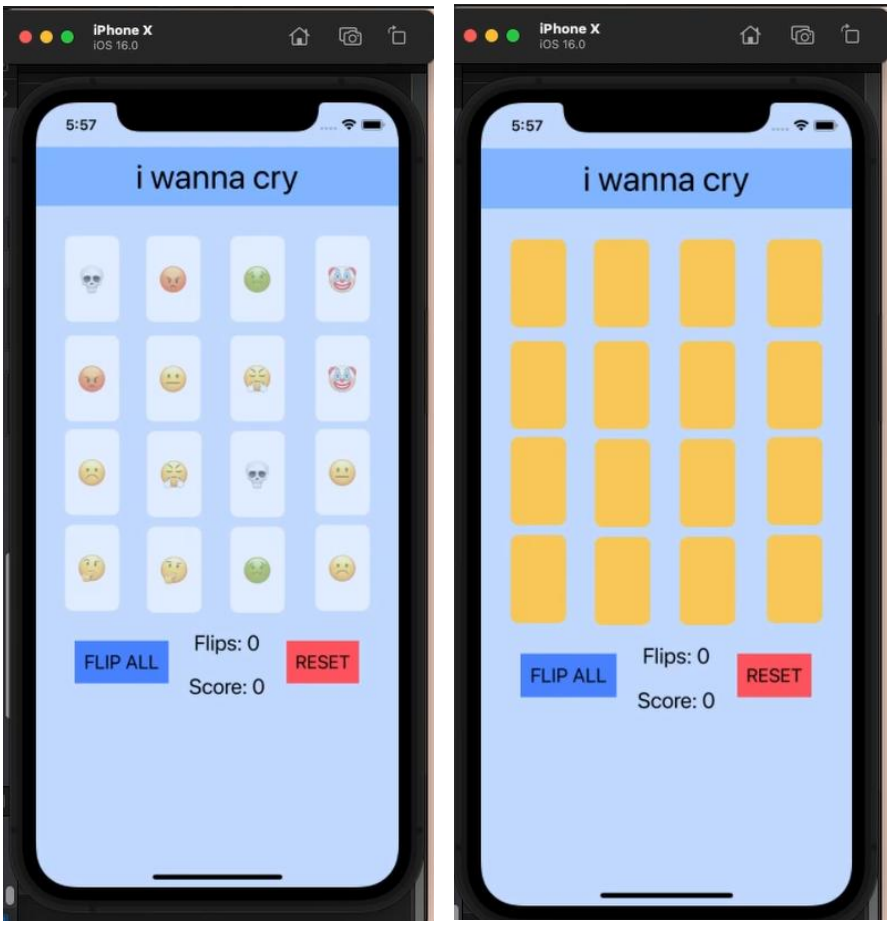
翻牌(兩張不同: 分數減一，兩張相同: 分數加二)


iPhone X
iOS 16.0
5:57
i wanna cry
FLIP ALL
Flips: 5
Score: 1
RESET
iPhone X
iOS 16.0
5:57
i wanna cry
FLIP ALL
Flips: 6
Score: 0
RESET

翻同樣的牌(分數減一)



RESET：重置遊戲並將分數和翻牌次數歸零



FLIP ALL: 將全部的牌翻開/蓋上

程式碼說明：

MatchingGame.swift:

```
class MatchingGame{
    var cards: Array<Card>
    var score = 0;
    var indexOfOneAndOnlyFaceUpCard: Int?{
        get{
            var foundIndex:Int?
            for index in cards.indices{
                if cards[index].isFaceUp && !cards[index].isMatched{
                    if foundIndex == nil{
                        foundIndex = index
                    }
                    else{
                        return nil
                    }
                }
            }
            return foundIndex
        }
        set(newValue){
            for i in cards.indices{
                if (!cards[i].isMatched){
                    cards[i].isFaceUp = (i == newValue)
                }
            }
        }
    }
```

變數: cards(所有的卡牌)、score(遊戲分數)、indexOfOneAndOnlyFaceUpCard(被選中的第一張牌的 index)

indexOfOneAndOnlyFaceUpCard: 當取得時，找到所有卡牌中被翻開但還沒有被配對的卡片的 index，當設定時，更新所有還沒被配對的卡牌的翻開狀態

```
func chooseCard(at index: Int){
    if !cards[index].isMatched{
        if let matchIndex = indexOfOneAndOnlyFaceUpCard, matchIndex != index{
            if cards[matchIndex].identifier == cards[index].identifier{
                cards[matchIndex].isMatched = true
                cards[index].isMatched = true
                score += 2;
            }
            else{
                score -= 1;
            }
            cards[index].isFaceUp = true
        }
        else if index == indexOfOneAndOnlyFaceUpCard && cards[index].isFaceUp{
            indexOfOneAndOnlyFaceUpCard = nil
            score -= 1;
        }
        else{
            if cards[index].isFaceUp{
                indexOfOneAndOnlyFaceUpCard = nil
            }
            else{
                indexOfOneAndOnlyFaceUpCard = index
            }
        }
    }
}
```

chooseCard: 首先確認選擇的卡片不是已經配對完成的，並分為三種情況:
1. 當選擇的第二張卡片與第一張不同時，判斷兩者的 Emoji 是否相同，相同的話將兩者設為已配對並讓分數加二，不同的話則分數減一
2. 當選擇的第二張卡片與第一張相同時，將卡片翻回去並分數減一
3. 還沒選擇過第一張卡片，將該卡片的 index 紀錄下來

```
init(pairs: Int){
    cards = Array<Card>()
    for _ in 1...pairs{
        let card = Card()
        cards.append(card)
        cards.append(card)
    }
}
```

建構子: 根據設定牌對數將兩張一樣的牌加入到 cards 中

Card.swift

```
import Foundation

struct Card{
    var isFaceUp = false
    var isMatched = false
    var identifier: Int
    static var identifierFactory = 0

    static func getUniqueIdentifier()->Int{
        identifierFactory+=1
        return identifierFactory
    }

    init(){
        self.identifier = Card.getUniqueIdentifier()
    }
}
```

變數: isFaceUp(是否被翻開)、isMatched(是否已配對)、identifier(紀錄為第幾種牌)

ViewController.swift

```
class ViewController: UIViewController {
    var emojiChoices = [
        ["😀","💀","🤔","🥶","🤢","😡","😟","😤"],
        ["🖕","☝️","🖐","✌️","🤞","👊","🤘","🖖"],
        ["🍏","🍎","🍋","🍊","🍌","🍉","🍇"],
    ]
    var emojiDictionary = [Int: String]()
    var flipCount = 0;
    var flipAllCount = 0;
    var theme = 0;
    var lastTheme: Int?
    lazy var game: MatchingGame = MatchingGame(pairs: 8)

    var numberOfPairsCard: Int{
        return (cards.count+1)/2
    }

    @IBOutlet weak var flipText: UILabel!
    @IBOutlet weak var scoreText: UILabel!
    @IBOutlet var cards: [UIButton]!
```

變數: emojiChoices(主題)、emojiDictionary(紀錄每一種牌對應的 Emoji)、flipCount(翻牌次數)、flipAllCount(按下 FLIP ALL 按鈕的次數)、theme(目前選擇的主題)、lastTheme(上一場遊戲的主題)、game(MatchingGame 本身)、numberOfPairsCard(牌對數)、flipText(翻牌次數文字)、scoreText(分數文字)、cards(卡片的按鈕)

```
@IBAction func touchCard(_ sender: UIButton) {
    if let id = cards.index(of: sender){
        if (!game.cards[id].isMatched){
            flipCount += 1
            game.chooseCard(at: id)
            updateViewFromModel()
        }
    }
    else{
        print("Not in the collection")
    }
    scoreText.text = "Score: \(game.score)"
    flipText.text = "Flips: \(flipCount)"
}
```

touchCard: 取得選擇卡片的 index，該卡片如果還沒被配對的話翻牌次數加一並讓 MatchingGame 處理選擇的卡片，最後更新卡片的顯示以及分數和翻牌的文字

```
func updateViewFromModel(){
    for index in cards.indices{
        let button = cards[index]
        let card = game.cards[index]
        let title = getEmoji(for: card)

        if !card.isFaceUp{
            button.setTitle("", for: UIControlState.normal)
            button.backgroundColor = #colorLiteral(red: 0.9686274529, green: 0.78039217, blue: 0.3450980484, alpha: 1)
            button.alpha = 1.0
        }
        else{
            button.setTitle(title, for: UIControlState.normal)
            button.backgroundColor = #colorLiteral(red: 1, green: 1, blue: 1, alpha: 1)

            if card.isMatched{
                button.alpha = 0.5
            }
            else{
                button.alpha = 1.0
            }
        }
    }
}
```

updateViewForModel: 更新所有卡牌的顯示，如果被翻開的話就顯示 Emoji，還沒翻開的話就顯示橘色背景並隱藏 Emoji，最後如果卡片已配對的話將卡片設為半透明

```
func getEmoji(for card: Card) -> String{
    if emojiDictionary[card.identifier] == nil, emojiChoices[theme].count > 0{
        let randomIndex = Int(arc4random_uniform(UInt32(emojiChoices[theme].count)))
        emojiDictionary[card.identifier] = emojiChoices[theme].remove(at: randomIndex)
    }
    return emojiDictionary[card.identifier] ?? "?"
}
```

getEmoji: 取得每一種卡片的 Emoji 是什麼，如果 emojiDictionary 還沒有紀錄該種卡片的 Emoji 是什麼的話就從目前主題的 Emojis 中隨機選擇一種還沒被選擇過的 Emoji 並記錄到 emojiDictionary 中

```
@IBAction func touchReset(_ sender: UIButton) {
    resetFunction()
}
```

touchReset: 當按下 RESET 按鈕時重置遊戲

```
func resetFunction(){
    game.score = 0
    scoreText.text = "Score: \(game.score)"

    flipCount = 0
    flipText.text = "Flips: \(flipCount)"

    var newTheme: Int
    repeat {
        newTheme = Int(arc4random_uniform(UInt32(emojiChoices.count)))
    } while newTheme == lastTheme

    theme = newTheme
    lastTheme = theme

    emojiDictionary = [Int: String]()
    emojiChoices = [
        ["😬","💀","😰","🤯","🤢","😡","😩","😤"],
        ["👆","☝️","🖐","✌️","👌","👊","🤘","🖖"],
        ["🍏","🍎","🍐","🍊","🍋","🍌","🍉","🍇"],
    ]

    for index in game.cards.indices{
        game.cards[index].isFaceUp = false
        game.cards[index].isMatched = false
        game.indexOfOneAndOnlyFaceUpCard = nil
    }
    var n = game.cards.count
    while n > 1 {
        n -= 1
        let k = Int(arc4random_uniform(UInt32(n + 1)))
        game.cards.swapAt(n, k)
    }

    updateViewFromModel()
```

resetFunction: 重置遊戲分數和翻牌次數、選擇跟上一場遊戲不同的 Emoji 主題並將 emojiDictionary 和 emojiChoices 重置、將所有卡片的狀態和第一張卡片的 index 紀錄重置、打亂所有的卡牌順序，最後更新遊戲的顯示

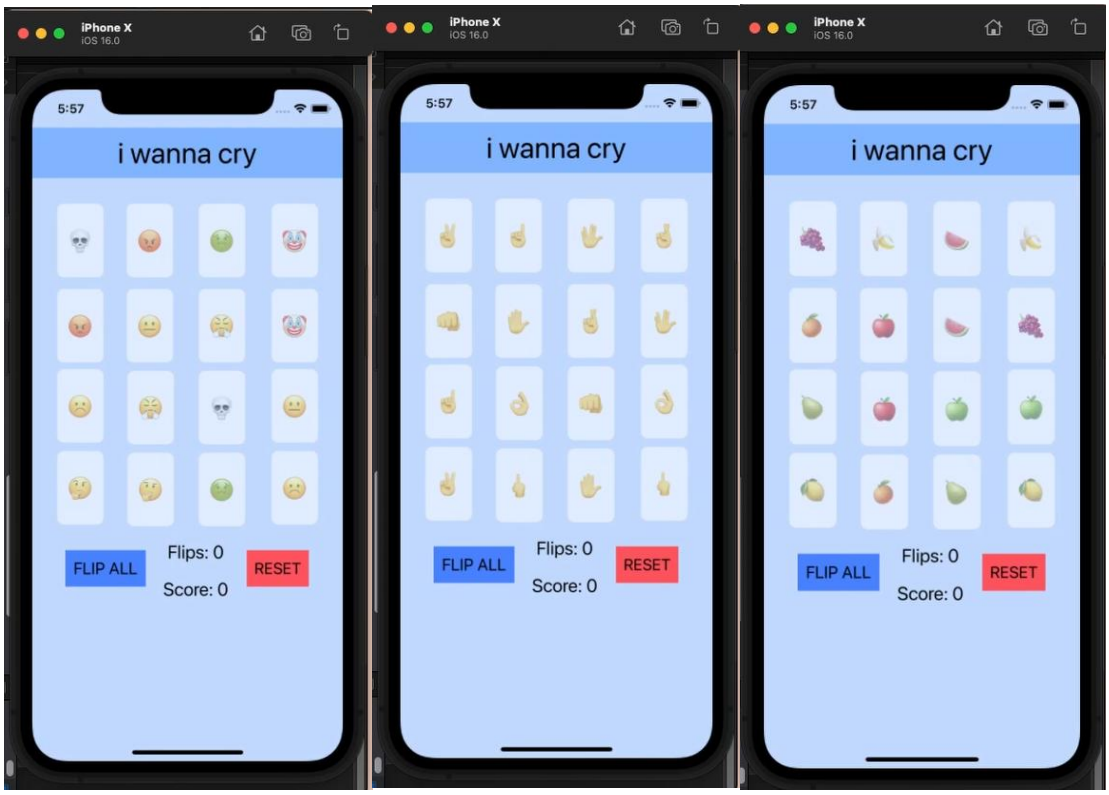
```
@IBAction func touchFlipAll(_ sender: UIButton) {
    flipCount = 0
    flipText.text = "Flips: \(flipCount)"
    game.score = 0
    scoreText.text = "Score: \(game.score)"

    if (flipAllCount % 2 == 0){
        for index in game.cards.indices{
            game.cards[index].isFaceUp = true
            game.cards[index].isMatched = true
        }
    }
    else{
        resetFunction()
    }
    game.indexOfOneAndOnlyFaceUpCard = nil
    flipAllCount += 1

    updateViewFromModel()
}
```

touchFlipAll: 當按下 FLIP ALL 按鈕時翻開所有的卡片並設定為已配對的狀態讓使用者無法按下任何卡片，再次按下時將遊戲重置

主題：



1.臉、2.手勢、3.水果